Q1 Solved

WAP to configure and read analog data from ADC channel no 5, and display the digital data on the LCD.

CODE

```
#include<LPC17xx.h>
#include<stdio.h>

void lcd_init(void);
void write(int, int);
void delay_lcd(unsigned int);
void lcd_comdata(int, int);
void clear_ports(void);
void lcd_puts(unsigned char *);

void lcd_init()
{
/*Ports initialized as GPIO */
LPC_PINCON->PINSEL1 &= 0xFC003FFF;  //P0.23 to P0.28
/*Setting the directions as output */
LPC_GPIO0->FIODIR |= 0x0F<<23 | 1<<27 | 1<<28;
clear_ports();
delay_lcd(3200);
        lcd_comdata(0x33, 0);
 delay_lcd(30000);
```

```
 lcd_comdata(0x32, 0);
 delay_lcd(30000);

 lcd_comdata(0x28, 0); //function set
 delay_lcd(30000);

 lcd_comdata(0x0c, 0);//display on cursor off
 delay_lcd(8000);

 lcd_comdata(0x06, 0); //entry mode set increment cursor right
 delay_lcd(8000);

 lcd_comdata(0x01, 0); //display clear
 delay_lcd(10000);

 return;
 }

void lcd_comdata(int temp1, int type)
{
 int temp2 = temp1 & 0xf0; //move data (26-8+1) times : 26 - HN  place, 4 - Bits
 temp2 = temp2 << 19;  //data lines from 23 to 26
 write(temp2, type);
 temp2 = temp1 & 0x0f;  //26-4+1
```

```
 temp2 = temp2 << 23;
 write(temp2, type);
 delay_lcd(10000);
 return;
}

void write(int temp2, int type)                    //write to command/data reg
 {
 clear_ports();
 LPC_GPIO0->FIOPIN = temp2; // Assign the value to the data lines
 if(type==0)
  LPC_GPIO0->FIOCLR = 1<<27; // clear bit RS for Command
else
  LPC_GPIO0->FIOSET = 1<<27; // set bit RS for Data

 LPC_GPIO0->FIOSET = 1<<28;    // EN=1
 delay_lcd(25);
 LPC_GPIO0->FIOCLR  = 1<<28; // EN =0
     return;
}

void delay_lcd(unsigned int r1)
{
   unsigned int r;
   for(r=0;r<r1;r++);
```

```
    return;
}

void clear_ports(void)
{
     /* Clearing the lines at power on */
 LPC_GPIO0->FIOCLR = 0x0F<<23; //Clearing data lines
 LPC_GPIO0->FIOCLR = 1<<27;  //Clearing RS line
 LPC_GPIO0->FIOCLR = 1<<28; //Clearing Enable line

    return;
}

void lcd_puts(unsigned char *buf1)
{
    unsigned int i=0;
 unsigned int temp3;
     while(buf1[i]!='\0')
     {
         temp3 = buf1[i];
        lcd_comdata(temp3, 1);
         i++;
         if(i==16)
         {
            lcd_comdata(0xc0, 0);
```

```
        }}return;}

#define Ref_Vtg 3.300
#define Full_Scale 0xFFF

int main(void)
{

unsigned long adc_temp;
unsigned int i;
float in_vtg;
unsigned char vtg[7], dval[7];
//12 bit ADC
unsigned char Msg3[11] = {"ANALOG IP:"};
unsigned char Msg4[12] = {"ADC OUTPUT:"};
SystemInit();
SystemCoreClockUpdate();
LPC_SC->PCONP |= (1<<15);    //Power for GPIO block
    lcd_init();
    LPC_PINCON->PINSEL3 |= 0xC0000000;  //P1.31 as AD0.5
 LPC_SC->PCONP |= (1<<12);   //enable the peripheral ADC

    SystemCoreClockUpdate();

 lcd_comdata(0x80, 0);
```

```
 delay_lcd(8000);
 lcd_puts(&Msg3[0]);

 lcd_comdata(0xC0, 0);
 delay_lcd(8000);
 lcd_puts(&Msg4[0]);

 while(1)
 {
LPC_ADC->ADCR = (1<<5)|(1<<21)|(1<<24); //0x01200001;
  //ADC0.5, start conversion and operational
  while(!(LPC_ADC->ADDR5 & 0x80000000));
        //wait till 'done' bit is 1, indicates conversion complete
  adc_temp = LPC_ADC->ADDR5;
  adc_temp >>= 4;
  adc_temp &= 0x00000FFF;    //12 bit ADC
          in_vtg = (((float)adc_temp * (float)Ref_Vtg))/((float)Full_Scale);
//calculating input analog voltage
  sprintf(vtg, "%3.2fV", in_vtg);
//convert the readings into string to display on LCD
  sprintf(dval, "%x", adc_temp);
  for(i=0; i<2000; i++);

  lcd_comdata(0x89, 0);
  delay_lcd(8000);
```

```
  lcd_puts(&vtg[0]);


  lcd_comdata(0xC8, 0);
  delay_lcd(8000);
  lcd_puts(&dval[0]);
        for(i=0;i<200000;i++);
for(i=0;i<7;i++)
vtg[i] = dval[i] = 0x00;
adc_temp = 0;
in_vtg = 0;
}
}
```

//Adjust delay value in the code and POT3 port on LPC board to get output according to your computer.

OUTPUT

C:\Users\ES_B2\Documents\B2_55\es lab10\lab10 solved\solved lab10.uvproj - µVision4

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Project

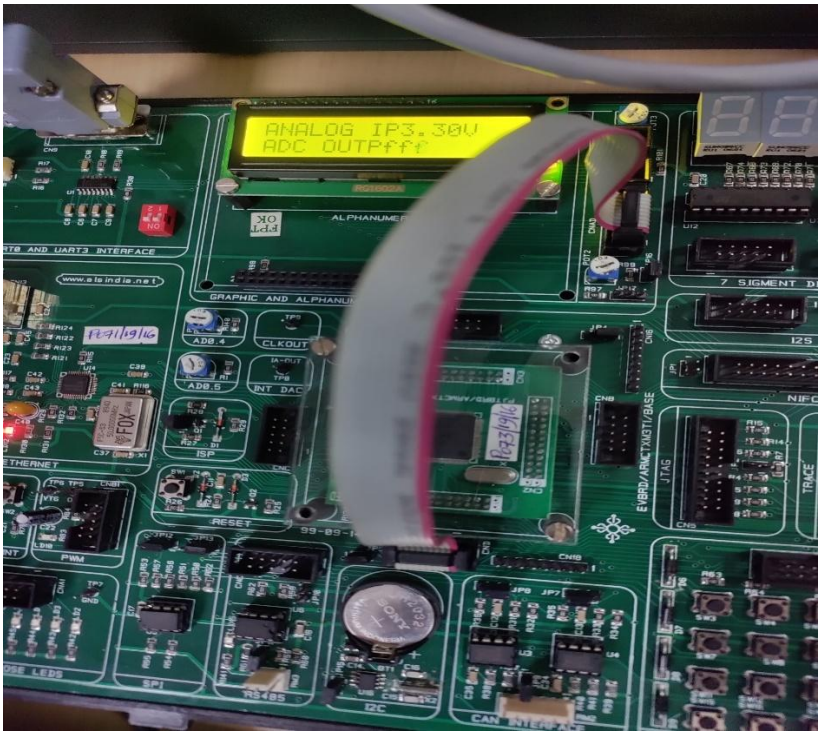
- Target 1
  - Source Group 1
    - startup_LPC17xx.s
    - system_LPC17xx.c
    - adc channel 5 to LCD

adc channel 5 to LCD.c

```c
        //wait till 'done' bit is 1, indicates conversion complete
  adc_temp = LPC_ADC->ADDR5;
  adc_temp >>= 4;
  adc_temp &= 0x00000FFF;    //12 bit ADC
  in_vtg = (((float)adc_temp * (float)Ref_Vtg))/((float)Full_Scale);
//calculating input analog voltage
  sprintf(vtg, "%3.2fV", in_vtg);
//convert the readings into string to display on LCD
  sprintf(dval, "%x", adc_temp);
  for(i=0; i<2000; i++);

  lcd_comdata(0x89, 0);
  delay_lcd(800);
  lcd_puts(&vtg[0]);

  lcd_comdata(0xC8, 0);
  delay_lcd(800);
  lcd_puts(&dval[0]);
  for(i=0;i<200000;i++);
for(i=0;i<7;i++)
vtg[i] = dval[i] = 0x00;
adc_temp = 0;
in_vtg = 0;
}
}
```

Build Output

```
Build target 'Target 1'
linking...
Program Size: Code=7452 RO-data=452 RW-data=4 ZI-data=612
FromELF: creating hex file...
".\solved lab10.axf" - 0 Error(s), 0 Warning(s).
```



As we vary the POT1(AD0.5) then the voltage value varies. Here we can see that the image shows 1.94V.

Q1 Write a C program to display the digital value representing the difference in analog voltages at ADC channel 4 and channel 5 on LCD using BURST and Software mode**.**

CODE

```
#include <lpc17xx.h>

#include <stdio.h>

void lcd_init(void);

void write(int, int);

void delay_lcd(unsigned int);

void lcd_comdata(int, int);

void clear_ports(void);

void lcd_puts(unsigned char *);


void lcd_init()

{

/*Ports initialized as GPIO */

LPC_PINCON->PINSEL1 &= 0xFC003FFF;  //P0.23 to P0.28

/*Setting the directions as output */

LPC_GPIO0->FIODIR |= 0x0F<<23 | 1<<27 | 1<<28;

clear_ports();
```

```
delay_lcd(38000);
        lcd_comdata(0x33, 0);
 delay_lcd(42000);

 lcd_comdata(0x32, 0);
 delay_lcd(42000);

 lcd_comdata(0x28, 0); //function set
 delay_lcd(42000);

 lcd_comdata(0x0c, 0);//display on cursor off
 delay_lcd(15000);

 lcd_comdata(0x06, 0); //entry mode set increment cursor right
 delay_lcd(15000);

 lcd_comdata(0x01, 0); //display clear
 delay_lcd(19000);

 return;
 }

void lcd_comdata(int temp1, int type)
{
 int temp2 = temp1 & 0xf0; //move data (26-8+1) times : 26 - HN  place, 4 - Bits
```

```
 temp2 = temp2 << 19;  //data lines from 23 to 26
 write(temp2, type);
 temp2 = temp1 & 0x0f;  //26-4+1
 temp2 = temp2 << 23;
 write(temp2, type);
 delay_lcd(19000);
 return;
}

void write(int temp2, int type)                    //write to command/data reg
 {
 clear_ports();
 LPC_GPIO0->FIOPIN = temp2; // Assign the value to the data lines
 if(type==0)
  LPC_GPIO0->FIOCLR = 1<<27; // clear bit RS for Command
else
  LPC_GPIO0->FIOSET = 1<<27; // set bit RS for Data

 LPC_GPIO0->FIOSET = 1<<28;    // EN=1
 delay_lcd(250);
 LPC_GPIO0->FIOCLR  = 1<<28; // EN =0
     return;
}

void delay_lcd(unsigned int r1)
```

```
{
   unsigned int r;
   for(r=0;r<r1;r++);
    return;
}

void clear_ports(void)
{
     /* Clearing the lines at power on */
 LPC_GPIO0->FIOCLR = 0x0F<<23; //Clearing data lines
 LPC_GPIO0->FIOCLR = 1<<27;  //Clearing RS line
 LPC_GPIO0->FIOCLR = 1<<28; //Clearing Enable line

    return;
}

void lcd_puts(unsigned char *buf1)
{
    unsigned int i=0;
 unsigned int temp3;
     while(buf1[i]!='\0')
     {
         temp3 = buf1[i];
        lcd_comdata(temp3, 1);
         i++;
```

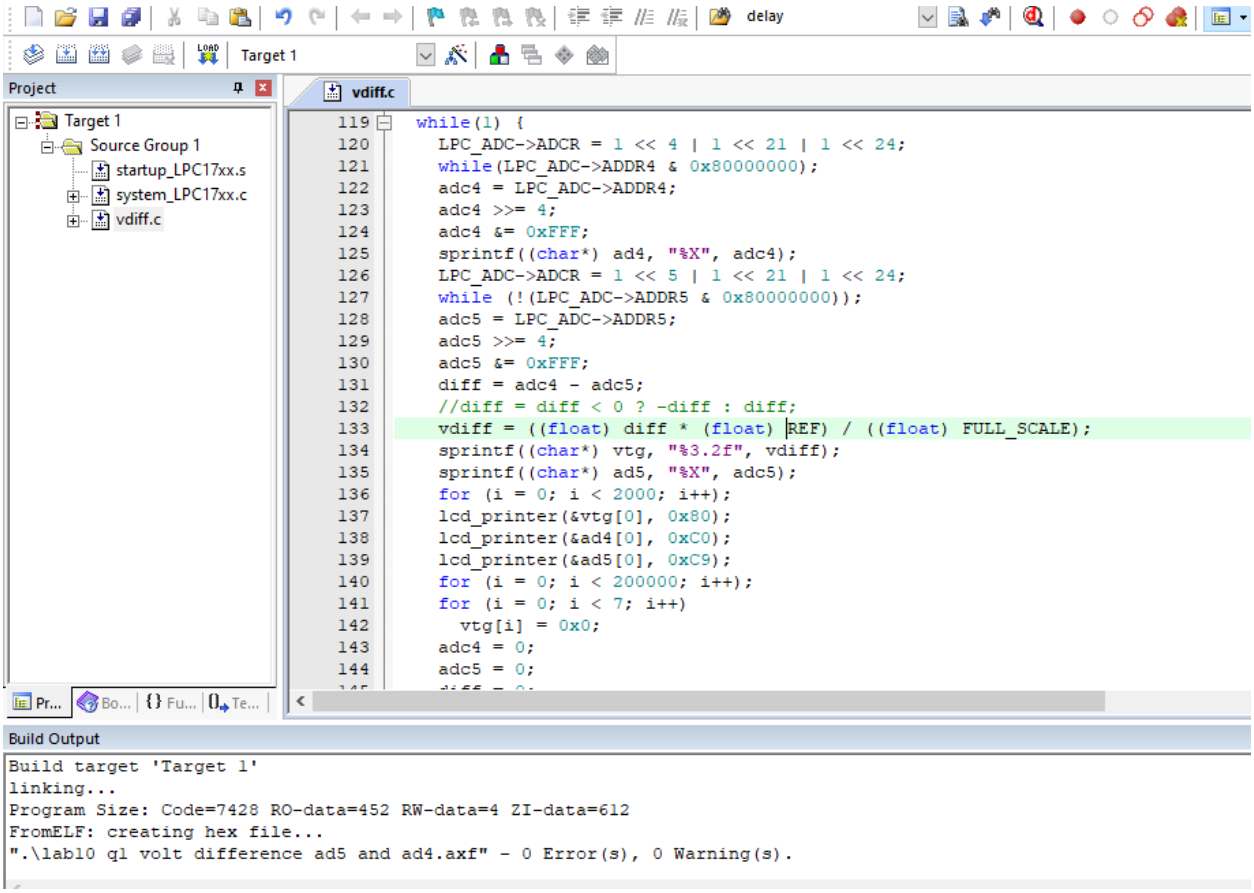
```
        if(i==16)
        {
          lcd_comdata(0xc0, 0);
        }}return;}



#define REF 3.300
#define FULL_SCALE 0xFFF

void lcd_printer(unsigned char* a, int screen_pos) {
        lcd_comdata(screen_pos, 0);
        delay_lcd(800);
        lcd_puts(a);
}

int main(void) {
        unsigned int adc4, adc5, i;
        int diff;
        float vdiff;
        unsigned char vtg[7],ad4[7],ad5[7];
        SystemInit();
        SystemCoreClockUpdate();
        LPC_SC->PCONP |= 1 << 15;
        lcd_init();
        LPC_PINCON->PINSEL3 |= 0xF0000000;
```

```
LPC_SC->PCONP |= 1 << 12;
SystemCoreClockUpdate();
while(1) {
	LPC_ADC->ADCR = 1 << 4 | 1 << 21 | 1 << 24;
	while(LPC_ADC->ADDR4 & 0x80000000);
	adc4 = LPC_ADC->ADDR4;
	adc4 >>= 4;
	adc4 &= 0xFFF;
	sprintf((char*) ad4, "%X", adc4);
	LPC_ADC->ADCR = 1 << 5 | 1 << 21 | 1 << 24;
	while (!(LPC_ADC->ADDR5 & 0x80000000));
	adc5 = LPC_ADC->ADDR5;
	adc5 >>= 4;
	adc5 &= 0xFFF;
	diff = adc4 - adc5;
	//diff = diff < 0 ? -diff : diff;
	vdiff = ((float) diff * (float) REF) / ((float) FULL_SCALE);
	sprintf((char*) vtg, "%3.2f", vdiff);
	sprintf((char*) ad5, "%X", adc5);
	for (i = 0; i < 2000; i++);
	lcd_printer(&vtg[0], 0x80);
	lcd_printer(&ad4[0], 0xC0);
	lcd_printer(&ad5[0], 0xC9);
	for (i = 0; i < 200000; i++);
	for (i = 0; i < 7; i++)
```

```
                vtg[i] = 0x0;
            adc4 = 0;
            adc5 = 0;
            diff = 0;
        }
}
```

//diff=adc4-adc5

Output



Burst

```
#include <lpc17xx.h>
#include <stdio.h>
```

```
void lcd_init(void);
void write(int, int);
void delay_lcd(unsigned int);
void lcd_comdata(int, int);
void clear_ports(void);
void lcd_puts(unsigned char *);

void lcd_init()
{
/*Ports initialized as GPIO */
LPC_PINCON->PINSEL1 &= 0xFC003FFF;  //P0.23 to P0.28
/*Setting the directions as output */
LPC_GPIO0->FIODIR |= 0x0F<<23 | 1<<27 | 1<<28;
clear_ports();
delay_lcd(42000);
        lcd_comdata(0x33, 0);
 delay_lcd(48000);

 lcd_comdata(0x32, 0);
 delay_lcd(48000);

 lcd_comdata(0x28, 0); //function set
 delay_lcd(48000);

 lcd_comdata(0x0c, 0);//display on cursor off
```

```
 delay_lcd(20000);

 lcd_comdata(0x06, 0); //entry mode set increment cursor right
 delay_lcd(20000);

 lcd_comdata(0x01, 0); //display clear
 delay_lcd(25000);

 return;
 }

void lcd_comdata(int temp1, int type)
{
 int temp2 = temp1 & 0xf0; //move data (26-8+1) times : 26 - HN  place, 4 - Bits
 temp2 = temp2 << 19;  //data lines from 23 to 26
 write(temp2, type);
 temp2 = temp1 & 0x0f;  //26-4+1
 temp2 = temp2 << 23;
 write(temp2, type);
 delay_lcd(19000);
 return;
}

void write(int temp2, int type)                    //write to command/data reg
 {
```

```
 clear_ports();
 LPC_GPIO0->FIOPIN = temp2; // Assign the value to the data lines
 if(type==0)
  LPC_GPIO0->FIOCLR = 1<<27; // clear bit RS for Command
else
  LPC_GPIO0->FIOSET = 1<<27; // set bit RS for Data

 LPC_GPIO0->FIOSET = 1<<28;    // EN=1
 delay_lcd(2500);
 LPC_GPIO0->FIOCLR  = 1<<28; // EN =0
     return;
}

void delay_lcd(unsigned int r1)
{
   unsigned int r;
   for(r=0;r<r1;r++);
    return;
}

void clear_ports(void)
{
     /* Clearing the lines at power on */
 LPC_GPIO0->FIOCLR = 0x0F<<23; //Clearing data lines
 LPC_GPIO0->FIOCLR = 1<<27;  //Clearing RS line
```

```
 LPC_GPIO0->FIOCLR = 1<<28; //Clearing Enable line

    return;
}

void lcd_puts(unsigned char *buf1)
{
    unsigned int i=0;
 unsigned int temp3;
     while(buf1[i]!='\0')
     {
         temp3 = buf1[i];
        lcd_comdata(temp3, 1);
         i++;
         if(i==16)
         {
            lcd_comdata(0xc0, 0);
         }}return;}



#define REF 3.300
#define FULL_SCALE 0xFFF

void lcd_printer(unsigned char* a, int screen_pos) {
```

```
	lcd_comdata(screen_pos, 0);
	delay_lcd(8000);
	lcd_puts(a);
}

int main(void) {
	unsigned int adc4, adc5, i;
	int diff;
	float vdiff;
	unsigned char vtg[7],ad4[7],ad5[7];
	SystemInit();
	SystemCoreClockUpdate();
	LPC_SC->PCONP |= 1 << 15;
	lcd_init();
	LPC_PINCON->PINSEL3 |= 0xF0000000;
	LPC_SC->PCONP |= 1 << 12;
	SystemCoreClockUpdate();
	while(1) {
		LPC_ADC->ADCR = 1 << 4 | 1 << 5 | 1 << 21 | 1 << 16;
		while(!(LPC_ADC->ADDR4 & 0x80000000) && !(LPC_ADC-
>ADDR5 & 0x80000000));
		adc4 = LPC_ADC->ADDR4;
		adc4 >>= 4;
		adc4 &= 0xFFF;
		sprintf((char*) ad4, "%X", adc4);
		adc5 = LPC_ADC->ADDR5;
```

```
            adc5 >>= 4;
            adc5 &= 0xFFF;
            diff = adc4 - adc5;
            //diff = diff < 0 ? -diff : diff;
            vdiff = ((float) diff * (float) REF) / ((float) FULL_SCALE);
            sprintf((char*) vtg, "%3.2f", vdiff);
            sprintf((char*) ad5, "%X", adc5);
            for (i = 0; i < 2000; i++);
            lcd_printer(&vtg[0], 0x80);
            lcd_printer(&ad4[0], 0xC0);
            lcd_printer(&ad5[0], 0xC9);
            for (i = 0; i < 200000; i++);
            for (i = 0; i < 7; i++)
                vtg[i] = 0x0;
            adc4 = 0;
            adc5 = 0;
            diff = 0;
        }
}
```